COMPREHENSIVE SYSTEM DESIGN AND IMPLEMENTATION CHALLENGE

Case Study: District Blood Bank Network

Subject	Comprehensive System Design and Implementation Challenge
Student Name	AKSHAYA R S
Register Number	1924260209
Course code	CSA0209
College	SIMATS Engineering College
Github link	https://github.com/akshaya12258-droid/CSA0209-C-PROGRAMMING1924260209

PART A – DESIGN & PSEUDOCODE (25 MARKS)

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION (25 MARKS)

Table of Contents

Section	Content
1	Case Study Introduction
2	PART A – Design & Pseudocode
2.1	Task A1 – Decompose the Program into Functions
2.2	Task A2 – Design with Recursion
2.3	Task A3 – Design with Pointers
2.4	Task A4 – Design Dynamic Memory Use
2.5	Task A5 – Design the File Storage
3	PART B – Implementation & Experimental Investigation
3.1	Task B1 – Implement and Instrument Recursive Functions
3.2	Task B2 – Engineer Adversarial Inputs
3.3	Task B3 – Text File vs Binary File
3.4	Task B4 – Final Engineering Decision
4	Final Engineering Conclusion
5	References

Navigation note: page numbers are generated by the PDF reader; use the section headings above to locate each task.

1. Case Study Introduction

The district blood bank network contains one central blood bank and several hospital storage units. The software must maintain donor details and blood-stock quantities, register donors, identify donors of a requested blood group, check donation eligibility, update stock after donations and issues, and preserve records in files.

Core data model. A donor record contains donor ID, name, blood group, age, phone number and last-donation date. A stock record contains a blood-group code and units available. The eight common ABO/Rh groups used in this design are O+, O−, A+, A−, B+, B−, AB+ and AB−.

Important design assumptions. The question does not prescribe a medical eligibility interval or all clinical rules. For the sample nightly report only, the program uses a configurable illustrative whole-blood interval of 56 days and reports donors who will have reached that interval by the end of the next seven days. This is a software demonstration, not medical advice or a replacement for clinical screening. The actual interval and age/health rules must be approved by the responsible blood-bank authority.

All C examples target ISO C11 and GCC. Unless explicitly labelled as a measured result, outputs are expected examples or theoretical calculations. Measurements in B3 were executed in one Linux environment and are identified as one-run observations, not universal performance guarantees.

2. PART A – DESIGN & PSEUDOCODE

Algorithmic pseudocode is language-independent: it describes the steps and decisions, not C syntax. The keywords BEGIN, END, INPUT, OUTPUT, IF, ELSE, WHILE, FOR, RETURN and FUNCTION are used consistently.

TASK A1 – DECOMPOSE THE PROGRAM INTO FUNCTIONS (7 MARKS)

A1.1 – Identify the Functions

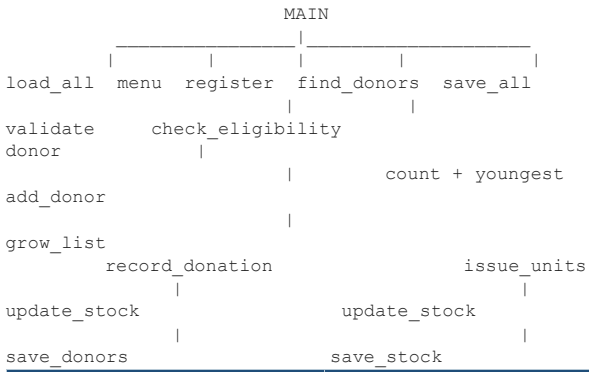
The main program coordinates smaller functions. Each function has one responsibility, making the program easier to understand, test and maintain.

Structure-level pseudocode

```
MAIN
  LOAD donor records from donor file
  LOAD stock records from stock file
  REPEAT
      DISPLAY menu
      INPUT user choice
      IF choice = REGISTER THEN
          CALL register_donor
      ELSE IF choice = FIND THEN
          CALL find_donors
      ELSE IF choice = DONATE THEN
          CALL record_donation
      ELSE IF choice = ISSUE THEN
          CALL issue_units
      ELSE IF choice = SAVE THEN
          CALL save_all
      END IF
  UNTIL choice = EXIT
  CALL save_all
  CALL release_donor_list
END
```

Structure chart (a line means “is called by”; helper functions may be reused):

Structure chart



Function	Responsibility
load_all	Open files, load donor and stock records, report read errors.
register_donor	Read and validate a new donor, then append to the list.
check_eligibility	Check required donor conditions and donation interval.
find_donors	Find eligible donors matching a requested blood group.
summarise_eligible	Return eligible count and youngest donor ID.
record_donation	Record a donation and increase the appropriate stock.
issue_units	Check available units and decrease stock after an issue.
update_stock	Find a stock record and apply a validated quantity change.
save_donors / save_stock	Write the latest data to persistent files.
grow_list / release_donor_list	Manage dynamically allocated donor memory safely.

A1.2 – Design the Interfaces

Assume these data types are declared before the prototypes:

```
#define NAME_LEN 50
#define PHONE_LEN 16
#define DATE_LEN 11

typedef struct {
int id;     char
name[NAME_LEN];
char group[4];     int
age;
    char phone[PHONE_LEN];
    char last_donation[DATE_LEN]; /* YYYY-MM-DD
*/ } Donor;

typedef struct {
int group_code;
int units; }
Stock;

typedef struct {
int count;
    int youngest_donor_id;
} EligibleSummary;

typedef struct {
Donor *data;
size_t size;
size_t capacity; }
DonorList;
```

C function prototypes

```
int load_donors(const char *filename, DonorList *list);
int load_stock(const char *filename, Stock stock[], size_t
stock_count); int register_donor(DonorList *list, const Donor
*new_donor); int check_eligibility(const Donor *donor, const char
*today); int find_donors(const DonorList *list, const char *group,
const char *today, int *matching_ids, size_t max_ids); int
summarise_eligible(const DonorList *list, const char *group,
const char *today, EligibleSummary *result); int
record_donation(DonorList *list, Stock stock[], size_t stock_count,
int donor_id, int units);
int issue_units(Stock stock[], size_t stock_count, int group_code, int
units); int update_stock(Stock *stock_record, int change); int
save_donors(const char *filename, const DonorList *list);
int save_stock(const char *filename, const Stock stock[], size_t
stock_count); int add_donor(DonorList *list, const Donor *donor); void
release_donor_list(DonorList *list);
```

Parameter form	Meaning and reason
int, size_t, char group[4] passed as scalar	Passed by value: small values are copied; the function cannot change the caller's scalar.
const Donor *donor	Pointer avoids copying the whole record. const means the function promises not to modify the donor through this pointer.
DonorList *list	Non-const pointer because size, capacity and data may change when a donor is added.
const DonorList *list	Read-only access to the list; prevents accidental modification during searching.
Stock stock[]	An array parameter is adjusted to a pointer. It is non-const when stock units must be updated.
const Stock stock[]	Read-only stock access when saving or reporting.
int *matching_ids	Output pointer: the function writes matching IDs into caller-owned storage.
EligibleSummary *result	Output pointer: allows a function to fill multiple result fields.
const char *filename / group / today	Read-only strings. The pointer itself is a local copy; characters must not be modified.

Pointer constness distinction. `const Donor *p` means the donor data cannot be changed through p. `Donor * const p` means p itself cannot be redirected, but donor data can be changed. `const Donor * const p` means neither the pointed-to donor nor the local pointer can be changed. In ordinary function parameters, the first form is useful for read-only input.

A1.3 – Return More Than One Value

The search must return two facts: the number of eligible donors and the ID of the youngest eligible donor. If no donor matches, use count = 0 and youngest_donor_id = -1.

Alternative 1: output pointer parameters.

```
FUNCTION summarise_eligible(list, group, today, count_out, youngest_id_out)
  SET count_out TO 0
  SET youngest_id_out TO -1
  SET youngest_age TO a very large value
  FOR EACH donor IN list
      IF donor.group = group AND donor is eligible on today THEN
          INCREMENT count_out
          IF donor.age < youngest_age THEN
              SET youngest_age TO donor.age
              SET youngest_id_out TO donor.id
          END IF
      END IF
  END FOR
  RETURN success
END FUNCTION
```

```
int summarise_eligible(const DonorList *list, const char
*group,                        const char *today, int
*count_out,                        int *youngest_id_out);
```

Alternative 2: return a structure. A C function can return one structure value containing several fields.

```
FUNCTION summarise_eligible_value(list, group, today)
  DECLARE result AS EligibleSummary
  SET result.count TO 0
  SET result.youngest_donor_id TO -1
  SCAN all donors and update both fields in result
  RETURN result
END FUNCTION
```

```
EligibleSummary summarise_eligible_value(const DonorList
*list,                                          const char
*group,                                          const char
*today);
```

Criterion	Pointer outputs	Returned structure
Readability	Several output parameters can make the call longer.	One named result object groups related outputs.
Safety	Caller must pass valid, non-null output addresses.	Returned value is self-contained; no output pointer can be null.
Extensibility	Adding another output changes the parameter list.	A new field can be added to the result structure.
Performance	Usually negligible difference; compiler handles small structures efficiently.	May copy a small structure, often optimized.
Best use	Useful when status and output buffers are already part of an API.	Best when outputs form one logical result.

Alternative concept: global variables and complexity

A global design could declare `DonorList donors` and `Stock stock[8]` at file scope. Functions would read and modify these variables without receiving them as parameters. Running time does not inherently change: scanning donors is still O(n), and updating a known stock entry is O(1). Memory also does not inherently change; the same donor and stock data must be stored somewhere.

However, global variables increase coupling: a function depends on hidden shared state. Tests must reset global data, independent copies of the function are harder to reuse, and unintended changes can affect distant functions. Passing explicit parameters makes dependencies visible and supports unit testing. Global state may be acceptable for a tiny demonstration, but is not preferred for this multi-hospital system.

TASK A2 – DESIGN WITH RECURSION (5 MARKS)

A2.1 – Recursive Design

Recursion occurs when a function solves a problem by calling itself on a smaller input. A base case stops the calls. A recursive case moves toward that base case.

Digital root. First add the decimal digits. If the result has more than one digit, repeat. The helper below sums the digits recursively; digital_root_recursive repeats that operation until one digit remains.

Recursive pseudocode (assume n is a non-negative integer)

```
FUNCTION SUM_DIGITS(n)
  IF n < 10 THEN
      RETURN n
  ELSE
      RETURN (n MOD 10) + SUM_DIGITS(n DIV 10)
  END IF
END FUNCTION

FUNCTION DIGITAL_ROOT(n)
  IF n < 10 THEN
      RETURN n
  ELSE
      SET total TO SUM_DIGITS(n)
      RETURN DIGITAL_ROOT(total)
  END IF
END FUNCTION
```

Power using divide-and-conquer. For a non-negative exponent, calculate the half power once. Square it, and multiply by base if the exponent is odd.

```
FUNCTION POWER_FAST(base, n)
  IF n = 0 THEN
      RETURN 1
  ELSE
      SET half TO POWER_FAST(base, n DIV 2)
      SET answer TO half * half
      IF n MOD 2 = 1 THEN
          SET answer TO answer * base
      END IF
      RETURN answer
  END IF
END FUNCTION
```

The linear version is POWER_LINEAR(base,n): if n = 0 return 1; otherwise return base × POWER_LINEAR(base,n−1).

A2.2 – Trace the Calls

Digital-root trace for 9875: SUM_DIGITS(9875) = 5 + SUM_DIGITS(987) = 5 + 7 + SUM_DIGITS(98) = 5 + 7 + 8 + SUM_DIGITS(9) = 29. Then DIGITAL_ROOT(29) → DIGITAL_ROOT(11) → DIGITAL_ROOT(2) → 2.

Call-stack trace

```
Deepest nested SUM_DIGITS call (top is the newest call):
  SUM_DIGITS(9)    -> base case returns 9
  SUM_DIGITS(98)   -> 8 + 9 = 17
  SUM_DIGITS(987) -> 7 + 17 = 24
  SUM_DIGITS(9875)-> 5 + 24 = 29
Then digital-root calls:
  DIGITAL_ROOT(9875) -> DIGITAL_ROOT(29)
  DIGITAL_ROOT(29)   -> DIGITAL_ROOT(11)
DIGITAL_ROOT(11)   -> DIGITAL_ROOT(2)
  DIGITAL_ROOT(2) returns 2
```

Power trace for POWER_FAST(2,5): calls descend 5 → 2 → 1 → 0. On return: POWER(2,0)=1; POWER(2,1)=1×1×2=2; POWER(2,2)=2×2=4; POWER(2,5)=4×4×2=32.

```
Deepest stack for POWER_FAST(2,5):
  POWER_FAST(2,0)  [base case; returns 1]
  POWER_FAST(2,1)  [waiting for half]
  POWER_FAST(2,2)  [waiting for half]
  POWER_FAST(2,5)  [original call]
Unwinding returns: 1 -> 2 -> 4 -> 32
```

Version	Call recurrence	Calls for exponent n	Time / stack
Linear power	C(0)=1; C(n)=C(n−1)+1	n+1	O(n) time; O(n) stack

Halving power	C(0)=1; C(n)=C(floor(n/2))+1	floor(log2 n)+2 for n≥1	O(log n) time; O(log n) stack

The call recurrence counts the current invocation, including the base-case call. For n=0, both versions make one call. For n≥1, halving calls are floor($\log_2$ n)+2.

A2.3 – Recursion Risks

Without a reachable base case, calls continue until the runtime cannot provide another stack frame; the program may terminate abnormally with stack overflow. Even with a base case, a very deep linear recursion can exhaust the stack. Recursion is useful when it makes a naturally recursive problem clearer, but a loop is preferable when the recursion depth is large, the operation is simple, or stack use must be tightly controlled.

Alternative: iterative and mathematical digital root

```
FUNCTION DIGITAL_ROOT_LOOP(n)
  WHILE n >= 10
      SET total TO 0
      WHILE n > 0
          total = total + (n MOD 10)
n = n DIV 10
      END WHILE
n = total
  END WHILE
  RETURN n
END FUNCTION
```

For a non-negative integer n, the mathematical formula is: digital_root(0)=0; otherwise digital_root(n)=1+((n−1) MOD 9). This is O(1) arithmetic operations for a fixed-width integer. The loop takes time proportional to the number of digits processed (O(d) for d digits, with repeated passes in the small digit-sum process) and uses O(1) extra space. Recursive digit summing uses stack space proportional to the number of digits in the current number; the outer repeated digital-root recursion adds further small stack depth. The formula uses O(1) stack space.

TASK A3 – DESIGN WITH POINTERS (5 MARKS)

A3.1 – Pointer Operations

Suppose the eight stock values are in `int stock[8]` and `int *p = stock;`. In an expression, the array name usually becomes a pointer to its first element.

Expression	Meaning
p + 3	Address of stock[3], three int elements after stock[0]. It does not change p.
*(p + 3)	Value stored in stock[3].
p[3]	Exactly equivalent to *(p + 3); value of stock[3].
*p++	Equivalent to *(p++): use the value pointed to by the old p, then advance p to the next int. It does not mean (*p)++.

Illustrative stock array (the addresses are symbolic; actual addresses depend on the run):

Memory diagram (assuming 4-byte int solely for illustration)

```
Blood group:  O+    O-    A+    A-    B+    B-    AB+   AB-
Index:         0     1     2     3     4     5     6     7
Value:        40    25    12     8    30    16     5    11
Address:    1000  1004  1008  1012  1016  1020  1024  1028
              ^ p initially points here p + 3 -> symbolic
address 1012 -> stock[3] -> value 8


After evaluating *p++: yielded 40; p now points to
stock[1].
```

A3.2 – Pointers and Functions

Safe C design

```
/* Returns NULL for an empty array or invalid input.
   The returned pointer refers to an element owned by the caller. */
int *lowest_stock(int stock[], size_t count)
{
    if (stock == NULL || count == 0)
return NULL;

    size_t lowest = 0;
    for (size_t i = 1; i < count; i++) {
if (stock[i] < stock[lowest])
lowest = i;
    }
    return &stock[lowest];
}

/* Caller:    int *p_low =
lowest_stock(stock, 8);    if (p_low
!= NULL) *p_low += 1; */
```

The function returns the address of an element inside the caller's array. The array remains alive after the function returns, so the pointer remains valid as long as the array remains alive and has not been moved or freed. Returning the address of a local variable is wrong because local automatic storage ends when the function returns; the pointer would dangle and dereferencing it would be undefined behaviour.

A3.3 – Command-Line Interface

Command examples

```
bloodbank find O+
bloodbank issue A- 2
bloodbank donate 7001 1
bloodbank report
```

The operating system passes the argument count in argc and the argument strings in argv. argv[0] is the program name. For `bloodbank issue A- 2`, argc is 4, argv[1] is "issue", argv[2] is "A-", and argv[3] is "2". The program must check argc before reading an argument, validate the command name, accept only one of the eight known groups, parse the unit count with strtol, reject non-numeric/negative/zero/excessive values, and display usage text for invalid input. It must not update stock if validation fails.

Alternative: return an index instead of a pointer

```
int lowest_stock_index(const int stock[], size_t count)
{
    if (stock == NULL || count == 0) return -1;
size_t lowest = 0;
    for (size_t i = 1; i < count; ++i)
if (stock[i] < stock[lowest]) lowest = i;
return (int)lowest;
}
/* Caller: int i = lowest_stock_index(stock, 8);
if (i >= 0) stock[i] += 1; */
```

Both versions scan n values: O(n) time and O(1) extra space. A pointer is convenient for direct updates but becomes invalid if the array is reallocated or freed. An index is a small integer and can be applied to the new array after reallocation, provided the logical element ordering has not changed. The index design is therefore safer across reallocations.

TASK A4 – DESIGN DYNAMIC MEMORY USE (4 MARKS)

A4.1 – Growable Donor List

A fixed array can be too small when the district grows. A dynamic array stores a pointer, current size and allocated capacity. Size is the number of donors currently stored; capacity is the number of records that fit before more memory is needed.

```
FUNCTION ADD_DONOR(list, donor)
  IF list.size = list.capacity THEN
      IF list.capacity = 0 THEN
new_capacity = 2
      ELSE
          new_capacity = 2 * list.capacity
      END IF
      SET temporary_pointer TO REALLOC(list.data,
                                       new_capacity * SIZE_OF(Donor))
      IF temporary_pointer = NULL THEN
          OUTPUT "Memory allocation failed"
          RETURN failure
      END IF
      list.data = temporary_pointer
list.capacity = new_capacity
  END IF
  list.data[list.size] = donor
list.size = list.size + 1
  RETURN success
END FUNCTION
```

In C, realloc may move the block. Always store its result in a temporary pointer first. If realloc fails, it returns NULL and the original block remains allocated; assigning NULL directly to list->data would lose the only pointer to the original block and cause a memory leak.

A4.2 – Memory Errors

Error	Blood-bank example	Prevention
Memory leak	Allocate a donor list, then overwrite its pointer without freeing the old block; or exit without freeing it.	Keep the owning pointer, handle realloc through a temporary pointer, and free the list once at shutdown.
Dangling pointer	Return a pointer to a local Donor, or keep a pointer to an element after realloc moved the array.	Return caller-owned storage only; after realloc, refresh element pointers. Set released pointers to NULL where useful.
Double free	Two cleanup paths both call free(list.data), or free the same block twice.	Define one owner and one cleanup path. After free, set data=NULL and size=capacity=0.

Alternative: grow by 10 versus doubling

With doubling, capacities progress 2, 4, 8, 16, … . The total number of old elements copied during all growth events is less than about 2n for n appended records (geometric series), so total append work is O(n) and amortized append cost is O(1). A single append that triggers realloc can still cost O(n). The unused capacity immediately after a growth is less than half the new capacity.

With fixed increments of 10, capacities progress 10, 20, 30, … . Reallocations can copy approximately 10+20+…+(n−10), which is $\Theta(n^2)$ total copying for n records. The maximum unused space after a growth is at most 9 records, but many more reallocations occur. Doubling is usually faster for large lists; fixed increments save some unused memory but scale poorly.

Strategy	Growth events	Total copied elements	Unused capacity
Doubling	O(log n)	O(n) overall	Less than about half of capacity after a growth
Add 10	O(n)	$O(n^2)$ overall	At most 9 records

TASK A5 – DESIGN THE FILE STORAGE (4 MARKS)

A5.1 – Text or Binary?

Donor file: text. Store one donor per line, for example: donor_id|name|group|age|phone|YYYY-MM-DD. Write with fprintf and read with fgets plus validated parsing. Text is human-readable, easy to inspect and can be moved between systems more easily. Before writing, reject or escape the delimiter and newline characters in names so one donor cannot accidentally become multiple records.

Donor text-file writing example

```
FILE *fp = fopen("donors.txt", "w");
if (fp == NULL) { perror("donors.txt"); return 0; } for
(size_t i = 0; i < list->size; ++i) {     const Donor
*d = &list->data[i];     if (fprintf(fp,
"%d|%s|%s|%d|%s|%s\n",                 d->id, d->name,
d->group, d->age,                d->phone, d-
>last_donation) < 0) {         perror("write
donors.txt"); fclose(fp); return 0;
    }
} if (fclose(fp) != 0) { perror("close donors.txt"); return
0; }
```

Stock file: binary. Each stock record has a fixed-size C structure. fwrite writes a record's bytes, and fread reads them. Since every record has the same size, the program can seek directly to record i. Binary is compact and convenient for fixed-record in-place updates.

Stock binary-file writing example

```
typedef struct { int group_code; int units; } Stock;
FILE *fp = fopen("stock.bin", "wb");
if (fp == NULL) { perror("stock.bin"); return 0; }
if (fwrite(stock, sizeof Stock, 8, fp) != 8) {
perror("write stock.bin"); fclose(fp); return 0;
}
if (fclose(fp) != 0) { perror("close stock.bin"); return 0; }
```

A5.2 – Update in Place

```
FUNCTION UPDATE_STOCK_FILE(file, record_index, new_units)
  IF record_index < 0 OR new_units < 0 THEN
      RETURN failure
  END IF
  position = record_index * SIZE_OF(Stock)
  IF FSEEK(file, position, SEEK_SET) fails THEN RETURN failure
  IF FREAD(one Stock record) does not read exactly one record THEN
      RETURN failure
  END IF
  change record.units to new_units
  IF FSEEK(file, -SIZE_OF(Stock), SEEK_CUR) fails THEN
      RETURN failure
  END IF
  IF FWRITE(one Stock record) does not write exactly one record THEN
      RETURN failure
  END IF
  RETURN success
END FUNCTION
```

A text record has variable length because names and other fields can have different lengths. Changing one line may make it longer or shorter; all following bytes would need to shift. Text files therefore usually require rewriting the file (or using a separate index and fixed-width records) to safely update a record.

Alternative: rewrite the text file

Read all n records, change the selected record in memory, then write all n records to a replacement file and atomically rename it after successful close. This is O(n) work per update. A fixed-size binary record can be updated at a known offset in O(1) record operations, ignoring storage-device latency. Text files are often larger because numbers and delimiters are stored as characters; binary size is approximately n × sizeof(Stock), plus any file metadata. Binary structures can contain padding and are not automatically portable across different compilers, architectures or endianness.

3. PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION

Engineering process: Implement → Instrument → Stress → Measure → Compare → Interpret. The code below is complete and C11-compatible. Compile with `gcc -std=c11 -Wall -Wextra file.c -o program`.
For memory checking, use AddressSanitizer where supported: `gcc -std=c11 -Wall -Wextra -fsanitize=address -g file.c -o program`.

Measurement integrity: B3 contains real measurements from a single run in the execution environment available while preparing this document. Other tasks provide reproducible instructions and theoretical counts; they do not claim a crash threshold, Valgrind output or sanitizer result that was not measured.

TASK B1 – IMPLEMENT AND INSTRUMENT THE RECURSIVE FUNCTIONS (7 MARKS)

B1.1 – Validate the Implementation

The following complete program implements recursive digital root, linear power and halving power. The counter increments once on entry to each power function. The exponent must be non-negative, and the chosen base/exponent must keep the mathematical result within long long range.

Complete C program: recursive functions

```
#include <stdio.h>

long long calls = 0;

long long sum_digits(long long n)
{
    if (n < 10) return n;
    return (n % 10) + sum_digits(n / 10);
}

long long digital_root(long long n)
{
    if (n < 0) return -1;          /* this program accepts n >= 0 */
    if (n < 10) return n;
    return digital_root(sum_digits(n));
}

long long power_linear(long long base, int exp)
{
calls++;
    if (exp == 0) return 1;
    return base * power_linear(base, exp - 1);
}

long long power_fast(long long base, int exp)
{
calls++;
    if (exp == 0) return 1;        /* TODO 1 completed: base case
*/     long long half = power_fast(base, exp / 2); /* compute once
*/     long long answer = half * half;     if (exp % 2 != 0)
answer *= base;     return answer; }

int main(void)
{
    printf("digital_root(0) = %lld\n", digital_root(0));
printf("digital_root(9875) = %lld\n", digital_root(9875));
printf("digital_root(99999) = %lld\n", digital_root(99999));

    calls = 0;
    printf("linear 2^10 = %lld; calls = ", power_linear(2, 10));
printf("%lld\n", calls);

    calls = 0;
    long long a = power_fast(2, 30);
    printf("fast 2^30 = %lld; calls = %lld\n", a, calls);

    calls = 0;
    printf("fast 7^0 = %lld; calls = %lld\n", power_fast(7, 0),
calls);     return 0; }
```

Expected values: digital_root(0)=0, digital_root(9875)=2, digital_root(99999)=9, 2^10=1024,

2^30=1073741824, and 7^0=1. Note: in C, the order in which printf evaluates separate arguments is not a safe way to read a counter changed by another argument expression. For accurate instrumentation, store the function result first, then print the counter. The program above does that for the fast 2^30 case; for a strict counter test, use the corrected test driver below.

```
/* Counter-safe test pattern */
calls = 0;
long long result = power_linear(2, 10);
printf("linear result=%lld calls=%lld\n", result, calls);

calls = 0;
result = power_fast(2, 30); printf("fast result=%lld
calls=%lld\n", result, calls);
```

Test plan and interpretation:

Input	Expected result	What it verifies
digital_root(0)	0	Zero is handled as a base case.
digital_root(9875)	2	Multiple digit-sum rounds: 9875→29→11→2.
digital_root(99999)	9	A repeated sum that stays a single digit after one round.
2^0	1	Power base case.
2^10	1024	Ordinary positive exponent.
2^30	1073741824	Matches the supplied expected output and fits long long.
7^1	7	Exponent one.
3^5	243	Odd exponent handling in halving version.

B1.2 – Compare with Part A

The C functions follow the pseudocode directly: both power versions stop at exp=0; the fast version computes the half result once, squares it and multiplies by base for an odd exponent. The C implementation adds a global call counter for measurement and a negative-input guard for digital_root. The call counter is instrumentation, not part of the mathematical algorithm.

B1.3 – Count the Calls

Exponent n	Linear calls n+1	Halving calls
10	11	5
100	101	8
1,000	1,001	11
10,000	10,001	15

The halving counts above include the base-case call: for n≥1, calls = floor($\log_2 n$)+2. The exact sequence for 10 is 10→5→2→1→0 (5 calls); for 100 it is 100→50→25→12→6→3→1→0 (8 calls); for 1,000 it takes 11 calls; for 10,000 it takes 15 calls. Linear calls are n+1 because the exponent decreases by one on each call.

Important instrumentation note. To count calls at n=10,000 without overflowing the numeric result, use base=1 so every return value remains 1. This isolates call-count behaviour from arithmetic overflow. A long linear recursion may still approach the system stack limit.

B1.4 – Find the Limit

Do not report one universal crash value. Stack size, compiler, optimization, operating system, instrumentation and frame size affect the limit. A safe experiment is to compile a separate program that uses base=1, increases the exponent gradually, runs each trial as a separate process, and records the largest completed exponent. Stop before repeatedly destabilising the environment. Compare linear recursion with a loop that uses constant stack space. The iterative version should not fail from recursive stack growth, although time and integer-range limits still apply.

Illustrative, not measured: a system might complete several thousand recursive calls but fail at a larger depth. This is only an example of variability; it is not a result for every computer.

Why calling power_fast twice is worse. If the code returns `power_fast(base, exp/2) * power_fast(base, exp/2)`, the same subproblem is calculated twice. Its recurrence becomes
T(n)=2T(floor(n/2))+O(1), which is O(n) calls for powers of two rather than O(log n). Computing half once stores the result and reuses it.

Iterative square-and-multiply: while exponent > 0, if it is odd multiply result by base; square base; divide exponent by 2. This takes O(log n) multiplications and O(1) stack space. Halving recursion also takes O(log n) time but uses O(log n) stack. Linear recursion takes O(n) time and O(n) stack.

Iterative square-and-multiply (valid when intermediate values fit long long)

```
long long power_iterative(long long base, unsigned int exp)
{
    long long result = 1;
while (exp > 0) {
        if (exp & 1U) result *= base;
exp >>= 1;
        if (exp > 0) base *= base;
    }
    return result;
}
```

TASK B2 – ENGINEER ADVERSARIAL INPUTS (5 MARKS)

Long donor names: scanf versus fgets

`scanf("%s", name)` without a width limit can write beyond the array when the input token is longer than the destination. This is a buffer overflow and undefined behaviour; it may corrupt nearby variables or crash. Even with a width limit, the remaining characters must be consumed or the next input operation may read the tail.

Safe and unsafe input patterns (include)

```
/* Unsafe demonstration ONLY: do not use with untrusted input.
*/ char name[20]; /* scanf("%s", name);  // unsafe: no maximum
width */

/* Width-limited scanf version */
if (scanf("%19s", name) != 1) {
    fprintf(stderr, "Could not read name\n");
}

/* Preferred line input for names containing spaces */
char line[100];
if (fgets(line, sizeof line, stdin) == NULL) {
fprintf(stderr, "Could not read name\n");
} else {
    size_t len = strlen(line);     if
(len > 0 && line[len - 1] == '\n')
line[len - 1] = '\0';     else {
        /* No newline: line may have been truncated. */
int ch;
        while ((ch = getchar()) != '\n' && ch != EOF) { }
fprintf(stderr, "Name too long; enter it again\n");
    }
}
```

For a deliberate overflow experiment, use a separate disposable test program compiled with AddressSanitizer and feed a token longer than the array. The sanitizer may report a stack-buffer-overflow. Do not use the unsafe version in the final application. fgets reads at most size−1 characters, adds a null terminator, and can preserve spaces.

Missing/read-only file and command-line errors

Complete CLI and file-error example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

static int valid_group(const char *g)
{
    const char *groups[] = {"O+", "O-", "A+", "A-", "B+", "B-", "AB+", "AB-
"};     for (size_t i = 0; i < 8; ++i)
        if (strcmp(g, groups[i]) == 0) return
1;     return 0; }

int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "Usage: %s find GROUP | issue GROUP UNITS\n", argv[0]);
return 2;
    }
    if (strcmp(argv[1], "find") == 0 && argc == 3) {
if (!valid_group(argv[2])) {
            fprintf(stderr, "Invalid blood group: %s\n", argv[2]);
return 2;
        }
        printf("Searching for group %s\n", argv[2]);
    } else if (strcmp(argv[1], "issue") == 0 && argc == 4)
{         char *end = NULL;         errno = 0;
        long units = strtol(argv[3], &end, 10);
        if (!valid_group(argv[2]) || errno != 0 || end == argv[3] ||
            *end != '\0' || units <= 0 || units > 1000000) {
fprintf(stderr, "Invalid group or unit count\n");
return 2;
        }
        printf("Issuing %ld unit(s) of %s\n", units, argv[2]);
    } else {
        fprintf(stderr, "Invalid arguments. Check command
usage.\n");         return 2;     }

    FILE *fp = fopen("donors.txt", "r");
if (fp == NULL) {
```

```
        perror("donors.txt");     /* prints reason, e.g. No such file */
return 1;
    }
fclose(fp);
return 0; }
```

Run and test:

```
gcc -std=c11 -Wall -Wextra cli.c -o cli
./cli
./cli find O+
./cli issue A- 2
./cli find X+
./cli issue A- two
./cli issue A- 2 extra
```

Test	Expected handling
No arguments	Usage message; non-zero exit status.
find O+	Accept valid group; then file open may report missing donors.txt.
issue A- 2	Accept valid command and positive integer.
find X+	Reject unknown blood group before file operation.
issue A- two	Reject non-numeric unit count.
Extra argument	Reject incorrect argc.

To test a missing file, run in a directory without donors.txt. To test read-only behaviour, make the file unreadable for the current user (permissions vary by OS); fopen("r") should fail and perror should explain why. To test write permission, open the file with "w" in a directory where writing is denied. Always check fopen, fread, fwrite, fseek and fclose return values. A read-only file opened with "r" is not an error; attempting to open it for writing is.

Memory safety tools and getline alternative

```
# AddressSanitizer build
gcc -std=c11 -Wall -Wextra -fsanitize=address -g cli.c -o cli_asan
./cli_asan find O+

# Valgrind (if installed)
gcc -std=c11 -Wall -Wextra -g cli.c -o cli valgrind --
leak-check=full --track-origins=yes ./cli find O+
```

No Valgrind or sanitizer run is claimed for this PDF unless the student performs it locally. A clean result should show no invalid reads/writes and no definitely-lost heap blocks. If an error appears, use the source line reported by the tool, correct ownership/bounds handling, and rerun.

getline alternative (POSIX extension, not ISO C11)

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>

int main(void)
{
    char *line = NULL;
size_t capacity = 0;
    ssize_t length = getline(&line, &capacity, stdin);
if (length >= 0)
        printf("Read %zd characters\n", length);
else
        perror("getline");
    free(line);       /* caller owns and must release the buffer
*/     return 0; }
```

For a line of length L, reading takes O(L) time and requires O(L) heap space (plus allocator overhead). getline grows the buffer as needed and updates both pointer and capacity. The caller must free the allocated buffer, even if getline fails after allocating it. On systems without getline, use a carefully implemented dynamic buffer or fgets with truncation detection.

TASK B3 – TEXT FILE VS BINARY FILE (8 MARKS)

B3.1 – Correctness on Small Data

The following complete program writes ten donor records to a delimited text file and to a binary file, reads both back, and compares every field. It also measures basic read/write durations and updates the 50,000th record when run with a sufficiently large record count. It uses a fixed test structure so the binary file has fixed-size records on the same compiled platform.

Complete text/binary comparison program (save as compare.c)

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

typedef struct {
int id;     char
name[40];     char
group[4];     int
age;     char
phone[16];
char date[11]; }
Donor;

static double seconds(void)
{
    struct timespec t;
timespec_get(&t, TIME_UTC);
    return (double)t.tv_sec + (double)t.tv_nsec /
1000000000.0; }

static void make_donor(Donor *d, int i)
{
    d->id = i + 1;
    snprintf(d->name, sizeof d->name, "Donor_%d", i +
1);     strcpy(d->group, (i % 2) ? "O+" : "A-");     d-
>age = 18 + i % 50;     strcpy(d->phone, "9876543210");
strcpy(d->date, "2026-01-01"); }

static int same(const Donor *a, const Donor *b)
{
    return a->id == b->id && strcmp(a->name,b->name)==0 &&
strcmp(a->group,b->group)==0 && a->age==b->age &&
           strcmp(a->phone,b->phone)==0 && strcmp(a->date,b-
>date)==0; }

static int write_text(const char *path, const Donor *a, size_t n)
{
    FILE *f = fopen(path, "w");     if
(!f) { perror(path); return 0; }
for (size_t i=0; i<n; ++i) {
        if (fprintf(f, "%d|%s|%s|%d|%s|%s\n", a[i].id,a[i].name,
a[i].group,a[i].age,a[i].phone,a[i].date) < 0) {
perror("fprintf"); fclose(f); return 0;
        }
    }
    if (fclose(f) != 0) { perror("fclose text"); return 0;
}     return 1; }

static int read_text(const char *path, Donor *a, size_t n)
{
    FILE *f=fopen(path,"r"); char
line[160];     if (!f) { perror(path);
return 0; }     for (size_t i=0; i<n; ++i)
{         if (!fgets(line,sizeof line,f)
||
            sscanf(line,"%d|%39[^|]|%3[^|]|%d|%15[^|]|%10s",
&a[i].id,a[i].name,a[i].group,&a[i].age,
a[i].phone,a[i].date) != 6) {
            fprintf(stderr,"Invalid text record %zu\n",i); fclose(f); return 0;
        }
    }
    fclose(f); return 1; } static int write_binary(const char
*path,const Donor *a,size_t n) {
    FILE *f=fopen(path,"wb"); if(!f){perror(path);return 0;}
if(fwrite(a,sizeof *a,n,f)!=n){perror("fwrite");fclose(f);return 0;}
if(fclose(f)!=0){perror("fclose binary");return 0;} return 1; }

static int read_binary(const char *path,Donor *a,size_t n)
{
```

```
    FILE *f=fopen(path,"rb"); if(!f){perror(path);return 0;}
size_t got=fread(a,sizeof *a,n,f); fclose(f);
    if(got!=n){fprintf(stderr,"Short binary file\n");return 0;} return
1; }

/* TODO 1 completed: update one fixed-size Stock record safely.
*/ typedef struct { int group_code; int units; } Stock; int
update_units(FILE *fp, int index, int units)
{
    Stock s;
    if (fp == NULL || index < 0 || units < 0) return 0;
if (fseek(fp, (long)index * (long)sizeof s, SEEK_SET) != 0)
return 0;
    if (fread(&s, sizeof s, 1, fp) != 1)
        return 0;                    /* invalid index: no write occurred */
s.units = units;
    if (fseek(fp, -(long)sizeof s, SEEK_CUR) != 0)
return 0;
    if (fwrite(&s, sizeof s, 1, fp) !=
1)         return 0;     return 1; }

int main(int argc,char **argv)
{
    size_t n = (argc==2) ? (size_t)strtoul(argv[1],NULL,10) : 10;
    if (n==0 || n>1000000) { fprintf(stderr,"Use 1..1000000 records\n"); return 2; }
    Donor *a=calloc(n,sizeof *a), *b=calloc(n,sizeof *b), *c=calloc(n,sizeof
*c);     if(!a||!b||!c){perror("calloc");free(a);free(b);free(c);return 1;}
for(size_t i=0;i<n;++i)make_donor(&a[i],(int)i);

    double t=seconds(); int ok=write_text("donors.txt",a,n); double tw=seconds()-t;
t=seconds(); ok=ok&&read_text("donors.txt",b,n); double tr=seconds()-t;
for(size_t i=0;ok&&i<n;++i) if(!same(&a[i],&b[i])) ok=0;

    t=seconds(); ok=ok&&write_binary("donors.bin",a,n); double bw=seconds()-
t;     t=seconds(); ok=ok&&read_binary("donors.bin",c,n); double
br=seconds()-t;     for(size_t i=0;ok&&i<n;++i) if(!same(&a[i],&c[i])) ok=0;
printf("records=%zu; text/binary identical=%s\n",n,ok?"YES":"NO");
printf("text write %.6f s; text read %.6f s\n",tw,tr);     printf("binary
write %.6f s; binary read %.6f s\n",bw,br);

    if(n>=50000){
        FILE *f=fopen("donors.bin","r+b");
if(!f){perror("donors.bin update");}
else {
            /* 50,000th human-numbered record has zero-based index 49,999.
*/             long offset=49999L*(long)sizeof(Donor);
            Donor d;
t=seconds();
            if(fseek(f,offset,SEEK_SET)==0 && fread(&d,sizeof d,1,f)==1){
                d.age++;
                if(fseek(f,-(long)sizeof d,SEEK_CUR)==0 &&
fwrite(&d,sizeof d,1,f)==1)
                    printf("binary record 50000 updated; time %.6f s\n",seconds()-t);
else perror("binary update write/seek");
            } else perror("binary update read/seek");
fclose(f);
        }
    }

free(a);free(b);free(c);
return ok?0:1; }
```

Compile and run: `gcc -std=c11 -Wall -Wextra compare.c -o compare`, then `./compare 10`. For scale tests run `./compare 1000`, `./compare 10000`, and `./compare 100000`. The program creates donors.txt and donors.bin in the current directory. Back up any files with those names before running.

For n=10, the correctness line must report YES if both files were written and read correctly. Compare all six fields, not merely the record count. The program's binary update uses zero-based index 49,999 for the 50,000th human-numbered record.

B3.2 – Increase the Data Size: measured results

Actual one-run environment: GCC 14.2.0, Debian Linux x86-64, 4,096-byte memory page. A separate optimized benchmark generated the same synthetic donor-like records (80-byte in-memory binary struct; text line with delimited fields), wrote and read each file, and measured one update of the 50,000th record at n=100,000. Times are wall-clock seconds from one run. Results vary with disk/cache, CPU load, filesystem, compiler and record layout.

Records	Text size (bytes)	Text write (s)	Text read (s)	Binary size (bytes)	Binary write (s)	Binary read (s)
1,000	41,786	0.000304	0.000021	80,000	0.000393	0.000033
10,000	437,788	0.001889	0.000147	800,000	0.001107	0.000221
100,000	4,577,790	0.036620	0.003986	8,000,000	0.009961	0.037751

In this particular record layout, binary files were larger because the binary struct occupied 80 bytes while the text records averaged about 42–46 bytes. This is a useful reminder: binary is not automatically smaller. Real structures may contain padding; text compression and field lengths also affect size. A production schema could use a compact binary encoding or a more efficient text representation.

B3.3 – Update One Record

Operation at 100,000 records	Measured time (s)	Work
Binary update of record 50,000	0.000039	Seek to fixed offset, read one record, seek back, write one record.
Text update by rewriting all records	0.061741	Read/parse all records, modify one in memory, rewrite entire text file.

The binary update time is effectively independent of record count for a fixed-size record when the record index is known: O(1) record operations. Rewriting a text file requires processing n records: O(n). These are measured elapsed times from one run, not guaranteed performance.

B3.4 – Engineering Trade-Off

Text is readable, easier to inspect and more portable as a representation. Binary is efficient for fixed-record access but a raw C structure is not a portable interchange format: compilers may add padding, integer widths can differ, and byte order can differ. A production binary format should define field widths, byte order, versioning and serialization explicitly.

Recommendation: keep donor master data in a validated text format or a database format designed for portability and indexing; keep rapidly updated fixed-size stock records in a structured binary/database table if the application controls the format. For a multi-hospital production network, a transactional database is generally safer than independent flat files because it supports concurrent updates, constraints, backups and audit trails.

Why record 9 fails safely

In the supplied TODO program, index 3 is valid because there are five records, so the fourth record is updated. Index 9 is beyond end-of-file. fread returns 0 before the program changes or writes a record; therefore the file remains unchanged. Always check the return count from fread before writing.

Alternative one-call fwrite: `fwrite(array, sizeof array[0], n, fp)` writes n contiguous records in one library call. Compared with n calls of fwrite per record, this reduces call overhead; the total data volume is still O(n). The entire array must be held in memory, requiring O(n) memory. A streaming approach can use O(1) record memory at the cost of repeated calls.

TASK B4 – FINAL ENGINEERING DECISION (5 MARKS)

B4.1 – Implement a Practical Strategy

The required add_donor TODO is completed below. The example preserves the supplied structure and demonstrates capacity doubling. A production donor record also needs name, age, phone and last-donation date; the report program that follows uses an expanded record.

Completed TODO 1: add_donor

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>

typedef struct { int id; char group[4]; } Donor;
typedef struct { Donor *data; size_t size, capacity; } DonorList;

int add_donor(DonorList *list, int id, const char *group)
{
    if (list == NULL || group == NULL) return 0;     if
(strlen(group) == 0 || strlen(group) > 3) return 0;

    if (list->size == list->capacity) {
        size_t new_capacity = (list->capacity == 0) ? 2 : list->capacity *
2;         if (new_capacity < list->capacity ||             new_capacity >
SIZE_MAX / sizeof(Donor))             return 0; /* multiplication/capacity
overflow guard */

        Donor *p = realloc(list->data, new_capacity * sizeof(Donor));
if (p == NULL) return 0;  /* old list->data remains valid */
list->data = p;
        list->capacity = new_capacity;
}

    list->data[list->size].id = id;
strcpy(list->data[list->size].group, group);
list->size++;     return 1; }

int main(void)
{
    DonorList list = {NULL, 0, 0};
    const char *groups[] = {"O+", "A+", "B-", "AB+", "O-"};
for (int i=0; i<5; ++i) {
        if (!add_donor(&list,7001+i,groups[i])) {
fprintf(stderr,"Could not add donor\n");
free(list.data); return 1;
        }
        printf("added %d (%s): size = %zu, capacity = %zu\n",
               list.data[i].id,list.data[i].group,list.size,list.capacity);
    }
    free(list.data);
    list.data=NULL; list.size=0;
list.capacity=0;     return 0; }
```

Expected output is the one in the question: capacities 2, 2, 4, 4, 8. realloc is stored in temporary pointer p because if realloc fails, it returns NULL while the original allocation remains valid. Assigning directly to list->data would overwrite the only reference to that old allocation and leak it.

Nightly eligibility report: complete example

This implementation demonstrates the growable-list approach using a donor text file with six pipe-separated fields: id|name|group|age|phone|last_donation. It checks the date interval only. It reports donors whose 56-day interval will be complete by the end of the next seven days (including those already due). The 56-day interval is an explicit demonstration assumption and must be replaced by the blood bank's approved policy.

Growable donor-list report program (save as report.c)

```
#define _XOPEN_SOURCE 700
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <stdint.h>

#define NAME_LEN 40
#define PHONE_LEN 16
#define DATE_LEN 11
#define DONATION_INTERVAL_DAYS 56
```

```
typedef struct {     int id;     char name[NAME_LEN];     char
group[4];     int age;     char phone[PHONE_LEN];     char
last_date[DATE_LEN]; } Donor; typedef struct { Donor *data;
size_t size, capacity; } DonorList;

static int add_donor(DonorList *list, const Donor *d)
{
    if (!list || !d) return 0;     if
(list->size == list->capacity) {
        size_t cap = list->capacity ? list->capacity * 2 : 2;
        if (cap < list->capacity || cap > SIZE_MAX / sizeof(Donor)) return
0;
        Donor *p = realloc(list->data, cap *
sizeof(Donor));         if (!p) return 0;         list-
>data = p;         list->capacity = cap;
    }
    list->data[list->size++] =
*d;     return 1; }

static int parse_date(const char *s, struct tm *out)
{     int
y,m,d;
char extra;
    if (sscanf(s,"%4d-%2d-%2d%c",&y,&m,&d,&extra) != 3) return 0;
memset(out,0,sizeof *out);
    out->tm_year=y-1900; out->tm_mon=m-1; out->tm_mday=d;     out-
>tm_hour=12; /* avoids many daylight-saving boundary problems */
out->tm_isdst=-1;     time_t value=mktime(out);     if
(value==(time_t)-1) return 0;
    /* Reject normalized invalid dates such as 2026-02-31. */      if (out-
>tm_year!=y-1900 || out->tm_mon!=m-1 || out->tm_mday!=d) return 0;     return
1; }

static int days_between(const char *date1, const char *date2, int *days)
{     struct tm
a,b;
    if (!parse_date(date1,&a) || !parse_date(date2,&b)) return
0;     time_t ta=mktime(&a), tb=mktime(&b);     if
(ta==(time_t)-1 || tb==(time_t)-1) return 0;

*days=(int)(difftime(tb,ta)/86400.0);
return 1; }

static int load_donors(const char *path, DonorList *list)
{
    FILE *fp=fopen(path,"r");
char line[256];
    if (!fp) { perror(path); return 0; }
while (fgets(line,sizeof line,fp)) {
        Donor d;
        if (sscanf(line,"%d|%39[^|]|%3[^|]|%d|%15[^|]|%10s",
                   &d.id,d.name,d.group,&d.age,d.phone,d.last_date)!=6)
{             fprintf(stderr,"Skipping malformed donor line\n");
continue;
        }
        if (!add_donor(list,&d)) {
            fprintf(stderr,"Memory allocation failed while loading\n");
fclose(fp); return 0;
        }
    }
    if (ferror(fp)) { perror("read donor file"); fclose(fp); return 0;
}     fclose(fp); return 1; }

int main(int argc,char **argv)
{
    const char *today = (argc==2) ? argv[1] : "2026-11-
11";     struct tm check;     if
(!parse_date(today,&check)) {
        fprintf(stderr,"Use a valid date YYYY-MM-DD\n"); return
2;     }

    DonorList list={NULL,0,0};     if
(!load_donors("donors.txt",&list)) {
free(list.data); return 1;
    }

    printf("Eligible within next 7 days (date basis
%s):\n",today);     for (size_t i=0;i<list.size;++i) {
int elapsed;
        if (days_between(list.data[i].last_date,today,&elapsed)
&&             elapsed >= DONATION_INTERVAL_DAYS-7) {
printf("%d | %s | %s | last donation %s\n",
list.data[i].id,list.data[i].name,
list.data[i].group,list.data[i].last_date);
```

```
        }
    }

    printf("Loaded %zu donors; capacity %zu; allocated bytes %zu\n",
list.size,list.capacity,list.capacity*sizeof(Donor));     free(list.data);
    list.data=NULL; list.size=0;
list.capacity=0;     return 0; }
```

Compile: `gcc -std=c11 -Wall -Wextra report.c -o report`. Run: `./report 2026-11-11`. Create donors.txt with the six-field format first. The date parser uses local time via mktime; for a production system, use a well-tested date library or store dates as validated calendar dates/UTC timestamps. This sample assumes a consistent local timezone for all records.

Sample input:

```
7001|Arun|O+|24|9876543210|2026-09-20
7002|Meena|A-|31|9876543211|2026-10-20
7003|Ravi|B+|28|9876543212|2026-08-01
```

With the illustrative reference date 2026-11-11, a donor whose last donation was on or before 2026-09-23 has at least 49 elapsed days and is reported as reaching the 56-day interval within seven days. Final eligibility must still be checked by qualified staff.

B4.2 – Check Memory Safety

The list owns one heap allocation. Every successful realloc updates the list pointer; failure leaves the old pointer intact. At shutdown, free(list.data) releases the allocation exactly once. Compile and run:

```
gcc -std=c11 -Wall -Wextra -fsanitize=address -g report.c -o report_asan
./report_asan 2026-11-11

# Optional Valgrind: gcc -std=c11 -Wall -Wextra -g report.c -o
report valgrind --leak-check=full --track-origins=yes ./report
2026-11-11
```

A clean AddressSanitizer/Valgrind report must be recorded only after the student actually runs the program. This document gives the commands and the ownership reasoning; it does not claim a clean tool report without an execution.

B4.3 – Production Decision

For 25 hospitals and a district-wide donor population, the decision depends on the measured size of the complete donor structure and available memory. Keeping all donors in a growable in-memory list gives O(1) access by array index and O(n) for a full eligibility scan, with O(n) memory. Reading in batches uses O(batch size) memory but a nightly scan still reads O(n) records. A separate sorted index file can support binary search by donor ID using O(log n) file reads, with O(1) working memory, but it adds index maintenance and consistency complexity.

Approach	Lookup/search	Memory	Advantages / drawbacks
All donors in memory	Array index O(1); full eligibility scan O(n)	O(n)	Fast repeated searches; simple report; needs RAM and reload time.
Read file in batches	Full scan O(n) I/O	O(batch size)	Low RAM; suitable for sequential nightly reports; slower random lookup.
Sorted index file	ID lookup O(log n) indexed reads	O(1) RAM plus index file	Low RAM and fast indexed lookup; index must be rebuilt/updated safely.

Illustrative memory calculation: with the expanded Donor structure shown in report.c, the exact sizeof(Donor) must be checked on the target compiler. If it is approximately 88 bytes and there are 1,000,000 donors, a capacity near 1,048,576 records uses about 92,274,688 bytes (approximately 88 MiB) for the donor array, excluding allocator overhead and other program memory. Use `sizeof(Donor)` and the actual capacity to calculate the exact value on the target computer. The report program prints allocated bytes as capacity × sizeof(Donor).

For a nightly report, a sequential batch scan is often sufficient if the report examines every donor once and RAM is constrained. If multiple interactive searches and reports are frequent and the measured donor array comfortably fits available RAM, loading the full list is simpler and faster. For a real 25-hospital network with concurrent updates, a central database with indexes and transactions is the recommended production solution; a flat-file prototype should not be treated as a multi-user transaction system.

Sorted-index-file alternative: store pairs (donor_id, byte_offset) sorted by donor ID. Binary-search the index, then fseek to the donor record. This requires O(log n) index reads for lookup and constant-size working memory if the index is streamed; caching the

index in RAM costs O(n) memory but makes the binary search much faster. The donor file must use fixed-size records or a reliable offset index. Every donor insert/delete/update must keep the index synchronized, preferably by rebuilding it safely or using a transactional database.

4. FINAL ENGINEERING CONCLUSION

The district blood-bank design is decomposed into functions with explicit interfaces, read-only pointers for input records, and structured result types for multiple outputs. Recursive digital root and halving power demonstrate base cases and call-stack behaviour; iterative alternatives reduce stack risk. Pointer-based stock updates are safe when returned pointers refer to caller-owned arrays, while indexes remain valid across array relocation. A doubling dynamic array provides amortized O(1) append operations and O(n) total growth-copy work. Text donor records are easy to inspect, while fixed-size binary stock records support direct in-place updates.

The experimental investigation demonstrates why measurement must accompany theory: binary data was faster for the measured update but larger for the chosen structure, and one-run read timings varied. The final system should validate all input, check every file operation, handle allocation failure, avoid unsafe string input, release memory once, and use a database or carefully designed transactional storage for a production multi-hospital network. Medical eligibility decisions must always follow the blood bank's approved clinical policy.

5. REFERENCES

No external web sources were used in preparing this assignment document. Technical conventions are based on the C11 language and standard C library interfaces described in the compiler's system headers and GCC toolchain documentation. For submission, the student may cite the prescribed course notes or textbook if required by the instructor.

Important final checks before submission: compile every code block saved as a .c file; run the commands on your own computer; replace illustrative outputs with your own screenshots and measured results where your instructor requires evidence; write your name and register number on the cover page; and confirm the institution's exact pseudocode template if it differs from the structured format used here.